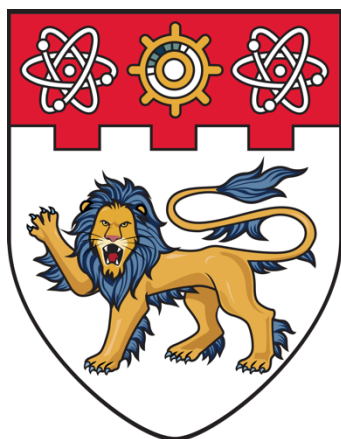




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

SC3000 Artificial Intelligence

Assignment 2

Name:	Goh Jun Keat
Matriculation Number:	U2320114E
Tutorial Group:	SCS2

Exercise 1: The Smart Phone Rivalry

sumsum, a competitor of appy, developed some nice smart phone technology called galaticas3, all of which was stolen by stevey, who is a boss of appy. It is unethical for a boss to steal business from rival companies. A competitor is a rival. Smart phone technology is business.

1. Translate the natural language statements above describing the dealing within the Smart Phone industry into First Order Logic (FOL).

Statement: "Sumsum is a competitor of appy."

FOL: competitor(sumsum, appy)

Statement: "Sumsum developed galaticas3."

FOL: develop(sumsum, galaticas3)

Statement: "Galaticas3 is a smart phone technology."

FOL: smart_phone_tech(galaticas3)

Statement: "Smart phone technology is business."

FOL: $\forall t, \text{smart_phone_tech}(t) \Rightarrow \text{business}(t)$

Statement: "Stevey stole galaticas3."

FOL: stole(stevey, galaticas3)

Statement: "Stevey is a boss of appy."

FOL: boss(stevey, appy)

Statement: "A competitor is a rival."

FOL: $\forall x, \forall y, \text{competitor}(x, y) \Rightarrow \text{rival}(x, y)$

Statement: "It is unethical for a boss to steal business from rival companies."

FOL: $\forall x, \forall y, \forall z, \forall t, \text{boss}(x, y) \wedge \text{rival}(y, z) \wedge \text{develop}(z, t) \wedge \text{business}(t) \wedge \text{stole}(x, t) \Rightarrow \text{unethical}(x)$

2. Write these FOL statements as Prolog clauses.

```
competitor(sumsum, appy).
develop(sumsum, galaticas3).
smart_phone_tech(galaticas3).
stole(stevey, galaticas3).
boss(stevey, appy).

business(T) :- smart_phone_tech(T).
rival(X, Y) :-
    competitor(X, Y);
    competitor(Y, X).

unethical(X) :-
    boss(X, Y),
    rival(Y, Z),
    develop(Z, T),
    business(T),
    stole(X, T).
```

3. Using Prolog, prove that Stevey is unethical. Show a trace of your proof.

```
?- trace, unethical(stevey).
Call: (13) unethical(stevey) ? creep
Call: (14) boss(stevey, _18356) ? creep
Exit: (14) boss(stevey, appy) ? creep
Call: (14) rival(appy, _20162) ? creep
Call: (15) competitor(appy, _20162) ? creep
Fail: (15) competitor(appy, _20162) ? creep
Redo: (14) rival(appy, _20162) ? creep
Call: (15) competitor(_20162, appy) ? creep
Exit: (15) competitor(sumsum, appy) ? creep
Exit: (14) rival(appy, sumsum) ? creep
Call: (14) develop(sumsum, _26484) ? creep
Exit: (14) develop(sumsum, galaticas3) ? creep
Call: (14) business(galaticas3) ? creep
Call: (15) smart_phone_tech(galaticas3) ? creep
Exit: (15) smart_phone_tech(galaticas3) ? creep
Exit: (14) business(galaticas3) ? creep
Call: (14) stole(stevey, galaticas3) ? creep
Exit: (14) stole(stevey, galaticas3) ? creep
Exit: (13) unethical(stevey) ? creep
true.
```

Exercise 2: The Royal Family

The old Royal succession rule states that the throne is passed down along the male line according to the order of birth before the consideration along the female line – similarly according to the order of birth. queen elizabeth, the monarch of United Kingdom, has four offsprings; namely:- prince charles, princess ann, prince andrew and prince edward – listed in the order of birth.

1. Define their relations and rules in a Prolog rule base. Hence, define the old Royal succession rule. Using this old succession rule determine the line of succession based on the information given. Do a trace to show your results.

a. Prolog Knowledge Base

```
male(prince_charles).
male(prince_andrew).
male(prince_edward).
female(princess_ann).
female(queen_elizabeth).

parent(queen_elizabeth, prince_charles).
parent(queen_elizabeth, princess_ann).
parent(queen_elizabeth, prince_andrew).
parent(queen_elizabeth, prince_edward).

older(prince_charles, princess_ann).
older(princess_ann, prince_andrew).
older(prince_andrew, prince_edward).

older(X, Z) :-
    older(X, Y),
    older(Y, Z).

compare_birth(Order, X, Y) :-
    older(X, Y) -> Order = '<';
    older(Y, X) -> Order = '>';
    Order = '='.

line_of_succession(Monarch, Succession) :-
    findall(X, (parent(Monarch, X), male(X)), Males),
    findall(Y, (parent(Monarch, Y), female(Y)), Females),
    psort(compare_birth, Males, SortedMales),
    psort(compare_birth, Females, SortedFemales),
    append(SortedMales, SortedFemales, Succession).
```

b. Trace

```
?- trace, line_of_succession(queen_elizabeth, Succession).
Call: (13) line_of_succession(queen_elizabeth, _16622) ? creep
^ Call: (14) findall(_18402, (parent(queen_elizabeth, _18402), male(_18402)), _18420) ? creep
^ Call: (19) parent(queen_elizabeth, _18402) ? creep
Exit: (19) parent(queen_elizabeth, prince_charles) ? creep
Call: (19) male(prince_charles) ? creep
Exit: (19) male(prince_charles) ? creep
Redo: (19) parent(queen_elizabeth, _18402) ? creep
Exit: (19) parent(queen_elizabeth, princess_ann) ? creep
Call: (19) male(princess_ann) ? creep
Fail: (19) male(princess_ann) ? creep
Redo: (19) parent(queen_elizabeth, _18402) ? creep
Exit: (19) parent(queen_elizabeth, prince_andrew) ? creep
Call: (19) male(prince_andrew) ? creep
Exit: (19) male(prince_andrew) ? creep
Redo: (19) parent(queen_elizabeth, _18402) ? creep
Exit: (19) parent(queen_elizabeth, prince_edward) ? creep
Call: (19) male(prince_edward) ? creep
Exit: (19) male(prince_edward) ? creep
^ Call: (14) findall(_18402, user:parent(queen_elizabeth, _18402), male(_18402)), [prince_charles, prince_andrew, prince_edward] ? creep
^ Call: (14) findall(_34722, (parent(queen_elizabeth, _34722), female(_34722)), _34740) ? creep
Call: (19) parent(queen_elizabeth, _34722) ? creep
Exit: (19) parent(queen_elizabeth, prince_charles) ? creep
Call: (19) female(prince_charles) ? creep
Fail: (19) female(prince_charles) ? creep
Redo: (19) parent(queen_elizabeth, _34722) ? creep
Exit: (19) parent(queen_elizabeth, princess_ann) ? creep
Call: (19) female(princess_ann) ? creep
Exit: (19) female(princess_ann) ? creep
Redo: (19) parent(queen_elizabeth, _34722) ? creep
Exit: (19) parent(queen_elizabeth, prince_andrew) ? creep
Call: (19) female(prince_andrew) ? creep
Fail: (19) female(prince_andrew) ? creep
Redo: (19) parent(queen_elizabeth, _34722) ? creep
Exit: (19) parent(queen_elizabeth, prince_edward) ? creep
Call: (19) female(prince_edward) ? creep
Fail: (19) female(prince_edward) ? creep
^ Exit: (14) findall(_34722, user:parent(queen_elizabeth, _34722), female(_34722)), [princess_ann] ? creep
^ Call: (14) sort:predsort(compare_birth, [prince_charles, prince_andrew, prince_edward], _51030) ? creep
Call: (17) compare_birth(_53912, prince_andrew, prince_edward) ? creep
Call: (18) older(prince_andrew, prince_edward) ? creep
Exit: (18) older(prince_andrew, prince_edward) ? creep
Call: (18) _53912=< ? creep
Exit: (18) {}=< ? creep
Exit: (17) compare_birth(<, prince_andrew, prince_edward) ? creep
Call: (17) compare_birth(_59346, prince_charles, prince_andrew) ? creep
Call: (18) older(prince_charles, prince_andrew) ? creep
Call: (19) older(prince_charles, _61156) ? creep
Exit: (19) older(prince_charles, princess_ann) ? creep
Call: (19) older(princess_ann, prince_andrew) ? creep
Exit: (19) older(princess_ann, prince_andrew) ? creep
Exit: (18) older(prince_charles, prince_andrew) ? creep
Call: (18) _164=< ? creep
Exit: (18) {}=< ? creep
Exit: (17) compare_birth(<, prince_charles, prince_andrew) ? creep
^ Call: (14) sort:predsort(user:compare_birth, [prince_charles, prince_andrew, prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
^ Call: (14) sort:predsort(compare_birth, [princess_ann], _5306) ? creep
Exit: (14) sort:predsort(user:compare_birth, [princess_ann], [princess_ann]) ? creep
Call: (14) lists:append([prince_charles, prince_andrew, prince_edward], [princess_ann], _58) ? creep
Exit: (14) lists:append([prince_charles, prince_andrew, prince_edward], [princess_ann], [prince_charles, prince_andrew, prince_edward, princess_ann]) ?
creep
Exit: (13) line_of_succession(queen_elizabeth, [prince_charles, prince_andrew, prince_edward, princess_ann]) ? creep
Succession = [prince_charles, prince_andrew, prince_edward, princess_ann].
```

2. Recently, the Royal succession rule has been modified. The throne is now passed down according to the order of birth irrespective of gender. Modify your rules and Prolog knowledge base to handle the new succession rule. Explain the necessary changes to the knowledge needed to represent the new information. Use this new succession rule to determine the new line of succession based on the same knowledge given. Show your results using a trace.

a. Prolog Knowledge Base

```
male(prince_charles).
male(prince_andrew).
male(prince_edward).
female(princess_ann).
female(queen_elizabeth).

parent(queen_elizabeth, prince_charles).
parent(queen_elizabeth, princess_ann).
parent(queen_elizabeth, prince_andrew).
parent(queen_elizabeth, prince_edward).

older(prince_charles, princess_ann).
older(princess_ann, prince_andrew).
older(prince_andrew, prince_edward).

older(X, Z) :-
    older(X, Y),
    older(Y, Z).

compare_birth(Order, X, Y) :-
    older(X, Y) -> Order = '<';
    older(Y, X) -> Order = '>';
    Order = '='.

new_line_of_succession(Monarch, Succession) :-
    findall(X, parent(Monarch, X), Children),
    predsort(compare_birth, Children, Succession).
```

b. Explanation

I removed the filtering based on gender from the original code. Instead of collecting separate lists of male and female children, all children of the monarch are now collected in 1 list. A person is a successor if the monarch is their parent, and they are included in the line of succession list ordered by their birth order alone.

c. Trace

```
?- trace, new_line_of_succession(queen_elizabeth, Succession).
Call: (13) new_line_of_succession(queen_elizabeth, _16622) ? creep
^ Call: (14) findall(_18404, parent(queen_elizabeth, _18404), _18412) ? creep
Call: (18) parent(queen_elizabeth, _18404) ? creep
Exit: (18) parent(queen_elizabeth, prince_charles) ? creep
Redo: (18) parent(queen_elizabeth, _18404) ? creep
Exit: (18) parent(queen_elizabeth, princess_ann) ? creep
Redo: (18) parent(queen_elizabeth, _18404) ? creep
Exit: (18) parent(queen_elizabeth, prince_andrew) ? creep
Redo: (18) parent(queen_elizabeth, _18404) ? creep
Exit: (18) parent(queen_elizabeth, prince_edward) ? creep
^ Call: (14) findall(_18404, user:parent(queen_elizabeth, _18404), [prince_charles, princess_ann, prince_andrew, prince_edward]) ? creep
Call: (17) compare_birth(_30406, prince_charles, princess_ann) ? creep
Call: (18) older(prince_charles, princess_ann) ? creep
Exit: (18) older(prince_charles, princess_ann) ? creep
Call: (18) _30406=(<) ? creep
Exit: (18) (<)=(<) ? creep
Exit: (17) compare_birth(<, prince_charles, princess_ann) ? creep
Call: (17) compare_birth(_35844, prince_andrew, prince_edward) ? creep
Call: (18) older(prince_andrew, prince_edward) ? creep
Exit: (18) older(prince_andrew, prince_edward) ? creep
Call: (18) _35844=(<) ? creep
Exit: (18) (<)=(<) ? creep
Exit: (17) compare_birth(<, prince_andrew, prince_edward) ? creep
Call: (17) compare_birth(_41278, prince_charles, prince_andrew) ? creep
Call: (18) older(prince_charles, prince_andrew) ? creep
Call: (19) older(prince_charles, _43088) ? creep
Exit: (19) older(prince_charles, princess_ann) ? creep
Call: (19) older(princess_ann, prince_andrew) ? creep
Exit: (19) older(princess_ann, prince_andrew) ? creep
Exit: (18) older(prince_charles, prince_andrew) ? creep
Call: (18) _41278=(<) ? creep
Exit: (18) (<)=(<) ? creep
Exit: (17) compare_birth(<, prince_charles, prince_andrew) ? creep
Call: (19) compare_birth(_50322, princess_ann, prince_andrew) ? creep
Call: (20) older(princess_ann, prince_andrew) ? creep
Exit: (20) older(princess_ann, prince_andrew) ? creep
Call: (20) _50322=(<) ? creep
Exit: (20) (<)=(<) ? creep
Exit: (19) compare_birth(<, princess_ann, prince_andrew) ? creep
^ Exit: (14) sort:predsort(user:compare_birth, [prince_charles, princess_ann, prince_andrew, prince_edward], [prince_charles, princess_ann, prince_andrew, prince_edward]) ? creep
Exit: (13) new_line_of_succession(queen_elizabeth, [prince_charles, princess_ann, prince_andrew, prince_edward]) ? creep
Succession = [prince_charles, princess_ann, prince_andrew, prince_edward].
```